
ЧАСТЬ II

БИБЛИОТЕКА C++

В этой части...

Глава 8. Библиотека ввода и вывода

Глава 9. Последовательные контейнеры

Глава 10. Обобщенные алгоритмы

Глава 11. Ассоциативные контейнеры

Глава 12. Динамическая память

С каждым выпуском новой версии языка C++ росла также его библиотека. На самом деле библиотеке посвящено больше двух третей текста нового стандарта. Хотя мы и не можем рассмотреть каждое средство библиотеки подробно, ее основные средства каждый программист C++ должен знать. Эти основные средства мы и рассмотрим в данной части.

Начнем в главе 8 с базовых средств библиотеки Ю. Кроме потоков чтения и записи, связанных с окном консоли, библиотека определяет типы, позволяющие читать и писать в именованные файлы и строки в оперативной памяти.

Основную часть библиотеки составляют многочисленные классы контейнеров и семейство обобщенных алгоритмов, позволяющих писать компактные и эффективные программы. Чтобы разработчик программы мог сосредоточиться на решении фактических проблем, библиотека берет на себя все подробности управления памятью.

В главе 3 мы познакомились с контейнером типа `vector`. Подробней мы рассмотрим его и другие типы последовательных контейнеров в главе 9, а также изучим больше операций, предоставленных типом `string`. Строку типа `string` можно считать специальным контейнером, который содержит только символы. Тип `string` поддерживает многие, но не все операции контейнеров.

В главе 10 представлены обобщенные алгоритмы. Обычно они работают с диапазоном элементов в последовательном контейнере или с другой последовательностью. Библиотека алгоритмов предоставляет эффективные реализации различных классических алгоритмов, такие как сортировка и поиск,

а также другие общие задачи. Например, есть алгоритм `copy`, который копирует элементы из одной последовательности в другую; алгоритм `find`, который ищет указанный элемент; и так далее. Алгоритмы обобщены двумя способами: они могут быть применены к различным видам последовательностей, и эти последовательности могут содержать элементы различных типов.

Библиотека предоставляет также несколько ассоциативных контейнеров, являющихся темой главы 11. Доступ к элементам в ассоциативном контейнере осуществляется по ключу. Ассоциативные контейнеры имеют много общих операций с последовательными контейнерами, а также определяют операции, являющиеся специфическими для ассоциативных контейнеров.

Завершается часть главой 12, рассматривающей средства управления динамической памятью, предоставляемые языком и библиотекой. В этой главе рассматриваются одни из самых важных новых библиотечных классов, являющихся стандартизированными версиями интеллектуальных указателей. Используя интеллектуальные указатели, можно сделать намного надежней код, который использует динамическую память. Эта глава завершается расширенным примером, в котором используются библиотечные средства, представленные во всей части II.

ГЛАВА 8

БИБЛИОТЕКА ВВОДА И ВЫВОДА

В этой главе...

8.1. Классы ввода-вывода	400
8.2. Ввод и вывод в файл	407
8.3. Строковые потоки	413
Резюме	417
Термины	417

Язык C++ не имеет дела с вводом и выводом непосредственно. Вместо этого ввод и вывод обрабатываются семейством типов, определенных в стандартной библиотеке. Они обеспечивают взаимосвязь с устройствами, файлами, окнами и консолью. Есть также типы, обеспечивающие ввод и вывод в оперативную память и строки.

Библиотека ввода и вывода определяет также операции чтения и записи значений встроенных типов. Кроме того, такие классы, как `string`, обычно определяют подобные операции ввода и вывода для работы с объектами данного класса.

В этой главе представлены основные принципы библиотеки IO. В последующих главах рассматриваются дополнительные возможности: создание собственных операторов ввода и вывода (глава 14), контроль формата и осуществление произвольного доступа к файлам (глава 17).

В предыдущих программах использовалось немало средств библиотеки IO, большинство из них было представлено в разделе 1.2 (стр. 30).

- Тип `istream` (input stream — поток ввода) обеспечивает операции ввода.
- Тип `ostream` (output stream — поток вывода) обеспечивает операции вывода.
- Объект `cin` класса `istream` читает данные со стандартного устройства ввода.
- Объект `cout` класса `ostream` записывает данные на стандартное устройство вывода.
- Объект `cerr` класса `ostream` записывает данные на стандартное устройство сообщений об ошибке. Объект `cerr`, как правило, используется для сообщений об ошибках в программе.

- Оператор `>>` используется для чтения данных, передаваемых в объект класса `istream`.
- Оператор `<<` используется для записи данных, передаваемых в объект класса `ostream`.
- Функция `getline()` (см. раздел 3.2.2, стр. 130) получает ссылку на объект класса `istream` и ссылку на объект класса `string`, а затем читает слово из потока ввода в строку.



8.1. Классы ввода-вывода

Типы и объекты ввода-вывода, которые мы использовали до сих пор, манипулируют символьными данными. По умолчанию эти объекты связаны с окном консоли пользователя. Конечно, реальные программы не могут быть ограничены вводом и выводом исключительно в окно консоли. Программам нередко приходится читать или писать в именованные файлы. Кроме того, операции ввода-вывода зачастую удобно использовать для обработки символов в строке. Приложениям также, вероятно, понадобится читать и писать на языках, которые требуют поддержки расширенных символов.

Для поддержки столь разных видов обработки ввода-вывода, кроме уже использованных ранее типов `istream` и `ostream`, библиотека определяет целую коллекцию типов ввода-вывода. Эти типы (табл. 8.1) определены в трех отдельных заголовках: заголовок `iostream` определяет базовые типы, используемые для чтения и записи в поток, заголовок `fstream` определяет типы, используемые для чтения и записи в именованные файлы, заголовок `sstream` определяет типы, используемые для чтения и записи в строки, расположенные в оперативной памяти.

Таблица 8.1. Типы и заголовки библиотеки ввода-вывода

Заголовок	Тип
<code>iostream</code>	<code>istream</code> , <code>wistream</code> — читают данные из потока <code>ostream</code> , <code>wostream</code> — записывают данные в поток <code>iostream</code> , <code>wiostream</code> — читают и записывают данные в поток
<code>fstream</code>	<code>ifstream</code> , <code>wifstream</code> — читают данные из файла <code>ofstream</code> , <code>wofstream</code> — записывают данные в файл <code>fstream</code> , <code>wfstream</code> — читают и записывают данные в файл
<code>sstream</code>	<code>istringstream</code> , <code>wistringstream</code> — читают данные из строки <code>ostringstream</code> , <code>wostringstream</code> — записывают данные в строку <code>stringstream</code> , <code>wstringstream</code> — читают и записывают данные в строку

Для поддержки языков, использующих расширенные символы, библиотека определяет набор типов и объектов, манипулирующих данными типа `wchar_t` (см. раздел 2.1.1, стр. 62). Имена версий для расширенных символов начинаются с буквы `w`. Например, объекты `wcin`, `wcout` и `wcerr` соответствуют обычным объектам `cin`, `cout` и `cerr`, но для расширенных символов. Такие объекты определяются в том же заголовке, что и типы для обычных символов. Например, заголовок `fstream` определяет типы `ifstream` и `wifstream`.

Взаимоотношения между типами ввода и вывода

Концептуально ни вид устройства, ни размер символов не влияют на операции ввода-вывода. Например, оператор `>>` можно использовать для чтения данных из окна консоли, из файла на диске или из строки. Точно так же этот оператор можно использовать независимо от того, читаются ли символы типа `char` или `wchar_t`.

Используя *наследование* (inheritance), библиотека позволяет игнорировать различия между потоками различных видов. Подобно шаблонам (см. раздел 3.3, стр. 141), связанные наследованием классы можно использовать, не вникая в детали того, как они работают. Более подробная информация о наследовании в языке C++ приведена в главе 15 и в разделе 18.3 (стр. 1002).

Если говорить коротко, то наследование позволяет сказать, что некий класс происходит от другого класса. Обычно объект производного класса можно использовать так, как будто это объект класса, от которого он происходит.

Типы `ifstream` и `istream` происходят от класса `istream`. Таким образом, объект типа `ifstream` или `istream` можно использовать так, как будто это объект класса `istream`. Объекты этих типов можно использовать теми же способами, что и объект `cin`. Например, можно вызвать функцию `getline()` объекта `ifstream` или `istream` либо использовать их оператор `>>` для чтения данных. Точно так же типы `ofstream` и `ostream` происходят от класса `ostream`. Следовательно, объекты этих типов можно использовать теми же способами, что и объект `cout`.



Все, что рассматривается в остальной части этого раздела, одинаково применимо как к простым, файловым и строковым потокам, а также к потокам для символов типа `char` или `wchar_t`.



8.1.1. Объекты ввода-вывода не допускают копирования и присвоения

Как упоминалось в разделе 7.1.3 (стр. 340), объекты ввода-вывода не допускают копирования и присвоения:

```
ofstream out1, out2;  
out1 = out2; // ошибка: нельзя присваивать потоковые объекты  
ofstream print(outstream); // ошибка: нельзя инициализировать параметр
```

```

out2 = print(out2);           // типа ostream
                             // ошибка: нельзя копировать потоковые объекты

```

Поскольку объекты типа ввода-вывода нельзя копировать, не может быть параметра или типа возвращаемого значения одного из потоковых типов (см. раздел 6.2.1, стр. 277). Функции, осуществляющие ввод-вывод, получают и возвращают поток через ссылки. Чтение или запись в объект ввода-вывода изменяет его состояние, поэтому ссылка не должна быть константой.

8.1.2. Флаги состояния

В связи с наследованием классов ввода-вывода возможно возникновение ошибок. Некоторые из ошибок исправимы, другие происходят глубоко в системе и не могут быть исправлены в области видимости программы. Классы ввода-вывода определяют функции и флаги, перечисленные в табл. 8.2, позволяющие обращаться к *флагам состояния* (condition state) потока и манипулировать ими.

Таблица 8.2. Флаги состояния библиотеки ввода-вывода

<code>strm::iostate</code>	<code>strm</code> — один из типов ввода-вывода, перечисленных в табл. 8.1 (стр. 400). <code>iostate</code> — машинно-зависимый целочисленный тип, представляющий флаг состояния потока
<code>strm::badbit</code>	Значение флага <code>strm::iostate</code> указывает, что поток недопустим
<code>strm::failbit</code>	Значение флага <code>strm::iostate</code> указывает, что операция ввода-вывода закончилась неудачей
<code>strm::eofbit</code>	Значение флага <code>strm::iostate</code> указывает, что поток достиг конца файла
<code>strm::goodbit</code>	Значение флага <code>strm::iostate</code> указывает, что поток не находится в недопустимом состоянии. Это значение гарантированно будет нулевым
<code>s.eof()</code>	Возвращает значение <code>true</code> , если для потока <code>s</code> установлен флаг <code>eofbit</code>
<code>s.fail()</code>	Возвращает значение <code>true</code> , если для потока <code>s</code> установлен флаг <code>failbit</code>
<code>s.bad()</code>	Возвращает значение <code>true</code> , если для потока <code>s</code> установлен флаг <code>badbit</code>
<code>s.good()</code>	Возвращает значение <code>true</code> , если поток <code>s</code> находится в допустимом состоянии
<code>s.clear()</code>	Возвращает все флаги потока <code>s</code> в допустимое состояние
<code>s.clear(флаг)</code>	Устанавливает определенный флаг (флаги) потока <code>s</code> в допустимое состояние. Флаг имеет тип <code>strm::iostate</code>
<code>s.setstate(флаг)</code>	Добавляет в поток <code>s</code> определенный флаг. Флаг имеет тип <code>strm::iostate</code>
<code>s.rdstate()</code>	Возвращает текущее состояние потока <code>s</code> как значение типа <code>strm::iostate</code>

В качестве примера ошибки ввода-вывода рассмотрим следующий код:

```
int ival;  
cin >> ival;
```

Если со стандартного устройства ввода ввести, например, слово **Boo**, то операция чтения потерпит неудачу. Оператор ввода ожидал значение типа `int`, но получил вместо этого символ `B`. В результате объект `cin` перешел в состояние ошибки. Точно так же объект `cin` окажется в состоянии ошибки, если ввести символ конца файла.

Как только произошла ошибка, последующие операции ввода-вывода в этом потоке будут терпеть неудачу. Читать или писать в поток можно только тогда, когда он находится в неошибочном состоянии. Поскольку поток может оказаться в ошибочном состоянии, код должен проверять его, прежде чем использовать. Проще всего определить состояние потокового объекта — это использовать его в условии:

```
while (cin >> word)  
    // ok: операция чтения успешна ...
```

Условие оператора `while` проверяет состояние потока, возвращаемого выражением `>>`. Если данная операция ввода успешна, состояние остается допустимым и условие выполняется.

Опрос состояния потока

Использование потока в условии позволяет узнать только то, допустим ли он. Это ничего не говорит о случившемся. Иногда необходимо также узнать причину недопустимости потока. Например, действия после достижения конца файла, вероятно, будут отличаться от таковых после ошибки на устройстве ввода-вывода.

Библиотека ввода-вывода определяет машинно-зависимый целочисленный тип `iostate`, используемый для передачи информации о состоянии потока. Этот тип используется как коллекция битов, подобно переменной `quiz1` в разделе 4.8 (стр. 213). Классы ввода-вывода определяют четыре значения `constexpr` (разделе 2.4.4, стр. 103) типа `iostate`, представляющие конкретные битовые схемы. Эти значения используются для указания конкретных видов состояний ввода-вывода. Они используются с побитовыми операторами (см. раздел 4.8, стр. 210) для проверки или установки нескольких флагов за раз.

Флаг `badbit` означает отказ системного уровня, такой как неисправимая ошибка при чтении или записи. Как только флаг `badbit` установлен, использовать поток обычно больше невозможно. Флаг `failbit` устанавливается после исправимой ошибки, такой как чтение символа, когда ожидалось числовые данные. Как правило, такие проблемы вполне можно исправить и продолжить использовать поток. Достижение конца файла устанавливает флаги

и eofbit и failbit. Флаг goodbit, у которого гарантированно будет значение 0, не означает отказа в потоке. Если любой из флагов badbit, failbit или eofbit будет установлен, то оценивающее данный поток условие окажется ложным.

Библиотека определяет также набор функций для опроса состояния этих флагов. Функция good() возвращает значение true, если ни один из флагов ошибок не установлен. Функции bad(), fail() и eof() возвращают значение true, когда установлен соответствующий бит. Кроме того, функция fail() возвращает значение true, если установлен флаг badbit. Корректный способ определения общего состояния потока подразумевал бы использование функции good() или fail(). На самом деле код проверки потока в условии эквивалентен вызову !fail(). Функции bad() и eof() оповещают только о конкретной ошибке.

Управление флагами состояния

Функция-член rdstate() возвращает значение типа iostate, соответствующее текущему состоянию потока. Функция setstate() позволяет установить указанные биты состояния, чтобы указать возникшую проблему. Функция clear() перегружена (см. раздел 6.4, стр. 302): одна ее версия не получает никаких аргументов, а вторая получает один аргумент типа iostate.

Версия функции clear(), не получающая никаких аргументов, сбрасывает все биты отказа. После ее вызова функция good() возвращает значение true. Эти функции-члены можно использовать следующим образом:

```
// запомнить текущее состояние объекта cin
auto old_state = cin.rdstate();
cin.clear(); // сделать объект cin допустимым
process_input(cin); // использовать объект cin
cin.setstate(old_state); // вернуть объект cin в прежнее состояние
```

Версия функции clear(), получающая аргумент, ожидает значение типа iostate, представляющее новое состояние потока. Для сброса отдельного флага используется функция-член rdstate() и побитовые операторы, позволяющие создать новое желаемое состояние.

Например, следующий код сбрасывает биты failbit и badbit, а бит eofbit оставляет неизменным:

```
// сбросить биты failbit и badbit, остальные биты оставить неизменными
cin.clear(cin.rdstate() & ~cin.failbit & ~cin.badbit);
```

Упражнения раздела 8.1.2

Упражнение 8.1. Напишите функцию, получающую и возвращающую ссылку на объект класса istream. Функция должна читать данные из потока до тех пор, пока не будет достигнут конец файла. Функция должна выводить

прочитанные данные на стандартное устройство вывода. Перед возвращением потока верните все значения его флагов в допустимое состояние.

Упражнение 8.2. Проверьте созданную функцию, передав ей при вызове объект `cin` в качестве аргумента.

Упражнение 8.3. В каких случаях завершится следующий цикл `while`?

```
while (cin >> i) /* ... */
```

8.1.3. Управление буфером вывода

Каждый объект ввода-вывода управляет буфером, используемым для хранения данных, которые программа читает или записывает. Например, при выполнении следующего кода литеральная строка могла бы быть выведена немедленно или операционная система могла бы сохранить данные в буфере и вывести их позже:

```
os << "please enter a value: ";
```

Использование буфера позволяет операционной системе объединить несколько операций вывода данной программы на системном уровне в одну операцию. Поскольку запись в устройство может занять много времени, возможность операционной системы объединить несколько операций вывода в одну может существенно повысить производительность.

Существует несколько условий, приводящих к сбросу буфера, т.е. к фактической записи на устройство вывода или в файл.

- Программа завершается нормально. Все буфера вывода освобождаются при выходе из функции `main()`.
- В некий случайный момент времени буфер может оказаться заполненным. В этом случае перед записью следующего значения происходит сброс буфера.
- Сброс буфера можно осуществить явно, используя такой манипулятор, как `endl` (см. раздел 1.2, стр. 30).
- Используя манипулятор `unitbuf`, можно установить такое внутреннее состояние потока, чтобы буфер освобождался после каждой операции вывода. Для объекта `cerr` манипулятор `unitbuf` установлен по умолчанию, поэтому запись в него приводит к немедленному выводу.
- Поток вывода может быть связан с другим потоком. В таком случае буфер привязанного потока сбрасывается при каждом чтении или записи другого потока. По умолчанию объекты `cin` и `cerr` привязаны к объекту `cout`. Следовательно, чтение из потока `cin` или запись в поток `cerr` сбрасывает буфер потока `cout`.

Сброс буфера вывода

В приведенных ранее программах уже не раз использовался манипулятор `endl`, который записывает символ новой строки и сбрасывает буфер. Существуют еще два подобных манипулятора: `flush` и `ends`. Манипулятор `flush` используется для сброса буфера потока без добавления символов в вывод. Манипулятор `ends` добавляет в буфер нулевой символ, а затем сбрасывает его.

```
cout << "hi!" << endl; // выводит hi, новую строку и сбрасывает буфер
cout << "hi!" << flush; // выводит hi и сбрасывает буфер, ничего не
                        // добавляя
cout << "hi!" << ends; // выводит hi, нулевой символ и сбрасывает буфер
```

Манипулятор `unitbuf`

Если сброс необходим при каждом выводе, лучше использовать манипулятор `unitbuf`, который сбрасывает буфер потока после каждой записи. Манипулятор `nounitbuf` восстанавливает для потока использование обычного управляемого системой сброса буфера:

```
cout << unitbuf; // при любой записи буфер будет сброшен немедленно
// любой вывод сбрасывается немедленно, без всякой буферизации
cout << nounitbuf; // возвращение к обычной буферизации
```

ВНИМАНИЕ! ПРИ СБОЕ ПРОГРАММЫ БУФЕР НЕ СБРАСЫВАЕТСЯ

Буфер вывода *не сбрасывается*, если программа завершается аварийно. При сбое программы вполне вероятно, что выводимые ею данные могут остаться в буфере, ожидая вывода.

При попытке отладить аварийно завершающуюся программу необходимо гарантировать, что любой *подозрительный* вывод будет сброшен сразу. Программист может впустую потратить множество часов на отслеживание вовсе не того кода только потому, что фактически последний буфер вывода просто не сбрасывается.

Связывание потоков ввода и вывода

Когда поток ввода связан с потоком вывода, любая попытка чтения данных из потока ввода приведет к предварительному сбросу буфера, связанного с потоком вывода. Библиотечные объекты `cout` и `cin` уже связаны, поэтому оператор `cin >> ival;` заставит сбросить буфер, связанный с объектом `cout`.



В интерактивных системах потоки ввода и вывода обычно связаны. При выполнении программы это гарантирует, что приглашения к вводу будут отображены до того, как система перейдет к ожиданию ввода данных пользователем.

Существуют две перегруженные (см. раздел 6.4, стр. 302) версии функции `tie()`: одна не получает никаких аргументов и возвращает указатель на поток вывода, к которому в настоящее время привязан данный объект, если таковой вообще имеется. Функция возвращает пустой указатель, если поток не связан.

Вторая версия функции `tie()` получает указатель на объект класса `ostream` и связывает себя с ним. Таким образом, код `x.tie(&o)` связывает поток `x` с потоком вывода `o`.

Объект класса `istream` или `ostream` можно связать с другим объектом класса `ostream`:

```
cin.tie(&cout); // только для демонстрации: библиотека
                // автоматически связывает объекты cin и cout
// old_tie указывает на поток (если он есть), в настоящее время связанный
// с объектом cin
ostream *old_tie = cin.tie(nullptr); // объект cin больше не связан
// связь cin и cerr; не лучшая идея, поскольку объект cin должен быть
// привязан к объекту cout
cin.tie(&cerr); // чтение в cin сбрасывает объект cerr, а не cout
cin.tie(old_tie); // восстановление обычной связи между cin и cout
```

Чтобы связать данный поток с новым потоком вывода, функции `tie()` передают указатель на новый поток. Чтобы разорвать существующую связь, достаточно передать в качестве аргумента значение `0`. Каждый поток может быть связан одновременно только с одним потоком. Однако несколько потоков могут связать себя с тем же объектом `ostream`.



8.2. Ввод и вывод в файл

В заголовке `fstream` определены три типа, поддерживающие операции ввода и вывода в файл: класс `ifstream` читает данные из указанного файла, класс `ofstream` записывает данные в файл, класс `fstream` читает и записывает данные в тот же файл. Использование того же файла для ввода и вывода рассматривается в разделе 17.5.3 (стр. 956).

Эти типы поддерживают те же операции, что и описанные ранее объекты `cin` и `cout`. В частности, для чтения и записи в файлы можно использовать операторы ввода-вывода (`<<` и `>>`), можно использовать функцию `getline()` (см. раздел 3.2.2, стр. 130) для чтения из потока `ifstream`. Материал, изложенный в разделе 8.1 (стр. 400), относится также и к этим типам.

Кроме поведения, унаследованного от типа `iostream`, определенные в заголовке `fstream` типы имеют в дополнение члены для работы с файлами, связанными с потоком. Эти операции перечислены в табл. 8.3, они могут быть вызваны для объектов классов `fstream`, `ifstream` или `ofstream`, но не других типов ввода-вывода.

Таблица 8.3. Операции, специфические для типов заголовка *fstream*

<code>fstream fstrm;</code>	Создает несвязанный файловый поток. <i>fstream</i> — это один из типов, определенных в заголовке <i>fstream</i>
<code>fstream fstrm(s);</code>	Создает объект класса <i>fstream</i> и открывает файл по имени <i>s</i> . Параметр <i>s</i> может иметь тип <code>string</code> или быть указателем на символьную строку в стиле C (см. раздел 3.5.4, стр. 172). Эти конструкторы являются явными (см. раздел 7.5.4, стр. 383). Заданный по умолчанию режим файла зависит от типа <i>fstream</i>
<code>fstream fstrm(s, режим);</code>	Подобен предыдущему конструктору, но открывает файл <i>s</i> в указанном режиме
<code>fstrm.open(s)</code> <code>fstrm.open(s, режим)</code>	Открывает файл <i>s</i> и связывает его с потоком <i>fstrm</i> . Параметр <i>s</i> может иметь тип <code>string</code> или быть указателем на символьную строку в стиле C. Заданный по умолчанию режим файла зависит от типа <i>fstream</i> . Возвращает тип <code>void</code>
<code>fstrm.close()</code>	Закрывает файл, с которым связан поток <i>fstrm</i> . Возвращает тип <code>void</code>
<code>fstrm.is_open()</code>	Возвращает значение типа <code>bool</code> , указывающее, был ли связанный с потоком <i>fstrm</i> файл успешно открыт и не был ли он закрыт



8.2.1. Использование объектов файловых потоков

Когда необходимо читать или писать в файл, определяется объект *файлового потока* (file stream), который связывается с файлом. Каждый класс файлового потока определяет функцию-член `open()`, которая выполняет все системные операции, необходимые для поиска указанного файла и его открытия для чтения или записи.

При создании файлового потока можно (но не обязательно) указать имя файла. При предоставлении имени файла функция `open()` вызывается автоматически:

```
ifstream in(ifile); // создать объект ifstream и открыть указанный файл
ofstream out;       // файловый поток вывода, не связанный с файлом
```



Этот код определяет `in` как входной поток, инициализированный для чтения из файла, указанного строковым аргументом `ifile`. Код определяет `out` как поток вывода, который еще не связан с файлом. По новому стандарту имена файлов могут быть переданы как в переменной библиотечного типа `string`, так и в символьном массиве в стиле C

(см. раздел 3.5.4, стр. 172). Предыдущие версии библиотеки допускали только символьные массивы в стиле C.

Использование `fstream` вместо `istream&`

Как упоминалось в разделе 8.1 (стр. 401), объект производного типа можно использовать в тех местах, где ожидается объект базового типа. Благодаря этому факту функции, получающие ссылку (или указатель) на один из типов `istream`, могут быть вызваны от имени соответствующего типа `fstream` (или `sstream`). Таким образом, если имеется функция, получающая ссылку `ostream&`, то ее можно вызвать, передав объект типа `ofstream`, то же относится к ссылке `istream&` и типу `ifstream`.

Например, функции `read()` и `print()` (см. раздел 7.1.3, стр. 340) можно использовать для чтения и записи в именованный файл. В этом примере подразумевается, что имена файлов ввода и вывода передаются как аргументы функции `main()` (см. раздел 6.2.5, стр. 288):

```
ifstream input(argv[1]); // открыть файл транзакций продаж
ofstream output(argv[2]); // открыть файл вывода
Sales_data total;        // переменная для хранения текущей суммы
if (read(input, total)) { // прочитать первую транзакцию
    Sales_data trans;     // переменная для хранения данных следующей
                          // транзакции
    while(read(input, trans)) { // читать остальные транзакции
        if (total.isbn() == trans.isbn()) // проверить isbn
            total.combine(trans); // обновить текущую сумму
        else {
            print(output, total) << endl; // отобразить результат
            total = trans; // обработать следующую книгу
        }
    }
    print(output, total) << endl; // отобразить последнюю транзакцию
} else // ввода нет
    cerr << "No data?!" << endl;
```

Кроме использования именованных файлов, этот код практически идентичен версии программы сложения, приведенной на стр. 333. Важнейшая часть — вызов функций `read()` и `print()`. Этим функциям можно передать объекты типа `fstream`, хотя типами их параметров определены `istream&` и `ostream&` соответственно.

Функции-члены `open()` и `close()`

Когда определяется пустой объект файлового потока, вызвав функцию `open()`, его впоследствии можно связать с файлом:

```
ifstream in(ifile); // создать объект ifstream и открыть указанный файл
ofstream out;       // файловый поток вывода, не связанный ни с каким
                   // файлом
out.open(ifile + ".copy"); // открыть указанный файл
```

При неудаче вызова функции `open()` устанавливается бит `failbit` (см. раздел 8.1.2, стр. 402). Поскольку вызов функции `open()` может потерпеть неудачу, имеет смысл проверить ее успешность:

```
if (out) // проверить успешность вызова функции open
        // вызов успешен, файл можно использовать
```

Это подобно использованию объекта `cin` в условии. При неудаче вызова функции `open()` условие не выполняется и мы не будем пытаться использовать объект `in`.

Как только файловый поток будет открыт, он остается связанным с определенным файлом. На самом деле вызов функции `open()` для файлового потока, который уже открыт, приводит к установке бита `failbit`. Последующие попытки использования этого файлового потока потерпят неудачу. Чтобы связать файловый поток с другим файлом, необходимо сначала закрыть существующий файл. Как только файл закрывается, его можно открыть снова:

```
in.close(); // закрыть файл
in.open(ifile + "2"); // открыть другой файл
```

Если вызов функции `open()` успешен, поток устанавливается в такое состояние, что функция `good()` возвратит значение `true`.

Автоматическое создание и удаление

Рассмотрим программу, функция `main()` которой получает список файлов для обработки (см. раздел 6.2.5, стр. 288). У такой программы может быть следующий цикл:

```
// для каждого переданного программе файла
for (auto p = argv + 1; p != argv + argc; ++p) {
    ifstream input(*p); // создает input и открывает файл
    if (input) {         // если ошибки с файлом нет, обработать его
        process(input);
    } else
        cerr << "couldn't open: " + string(*p);
} // input выходит из области видимости и удаляется при каждой итерации
```

При каждой итерации создается новый объект класса `ifstream` по имени `input` и открывается файл для чтения. Как обычно, проверяется успех вызова функции `open()`. Если все в порядке, этот файл передается функции, которая будет читать и обрабатывать ввод. В противном случае выводится сообщение об ошибке.

Поскольку объект `input` является локальным для цикла `while`, он создается и удаляется при каждой итерации (см. раздел 5.4.1, стр. 247). Когда объект `fstream` выходит из области видимости, файл, к которому он привязан, автоматически закрывается. На следующей итерации объект `input` создается снова.



Когда объект класса `fstream` удаляется, автоматически вызывается функция `close()`.

Упражнения раздела 8.2.1

Упражнение 8.4. Напишите функцию, которая открывает файл и читает его содержимое в вектор строк, сохраняя каждую строку как отдельный элемент вектора.

Упражнение 8.5. Перепишите предыдущую программу так, чтобы каждое слово сохранялось в отдельном элементе.

Упражнение 8.6. Перепишите программу книжного магазина из раздела 7.1.1 (стр. 333) так, чтобы читать транзакции из файла. Передавайте имя файла как аргумент функции `main()` (см. раздел 6.2.5, стр. 288).



8.2.2. Режимы файла

Каждый поток обладает *режимом файла* (`file mode`), определяющим возможный способ использования файла. Список режимов файла и их значений приведен в табл. 8.3.

Таблица 8.4. Режимы файла

<code>in</code>	Открывает файл для ввода
<code>out</code>	Открывает файл для вывода
<code>app</code>	Переходит в конец файла перед каждой записью
<code>ate</code>	Переходит в конец файла непосредственно после открытия
<code>trunc</code>	Усекает существующий поток при открытии
<code>binary</code>	Осуществляет операции ввода-вывода в бинарном режиме

Режим файла можно указать при каждом открытии файла, будь то вызов функции `open()` или косвенное открытие файла при инициализации потока именем файла. У режимов, которые можно задать, есть ряд ограничений.

- Режим `out` может быть установлен только для объектов типа `ofstream` или `fstream`.
- Режим `in` может быть установлен только для объектов типа `ifstream` или `fstream`.
- Режим `trunc` может быть установлен, только если устанавливается также режим `out`.

- Режим `app` может быть установлен, только если не установлен режим `trunc`. Если режим `app` установлен, файл всегда открывается в режиме вывода, даже если это не было указано явно.
- По умолчанию файл, открытый в режиме `out`, усекается, даже если не задан режим `trunc`. Чтобы сохранить содержимое файла, открытого в режиме `out`, необходимо либо задать также режим `app`, тогда можно будет писать только в конец файла, либо задать также режим `in`, тогда файл откроется и для ввода, и для вывода. Использование того же файла для ввода и вывода рассматривается в разделе 17.5.3 (стр. 956).
- Режимы `ate` и `binary` могут быть установлены для объекта файлового потока любого типа и в комбинации с любыми другими режимами.

Для каждого типа файлового потока задан режим файла по умолчанию, который используется в случае, если режим не задан. Файлы, связанные с потоками типа `istream`, открываются в режиме `in`; файлы, связанные с потоками типа `ostream`, открываются в режиме `out`; а файлы, связанные с потоками типа `fstream`, открываются в режимах `in` и `out`.

Открытие файла в режиме `out` удаляет существующие данные

По умолчанию при открытии потока типа `ostream` содержимое файла удаляется. Единственный способ воспрепятствовать удалению данных файла подразумевает установку режима `app`:

```
// file1 усекается в каждом из следующих случаев
ofstream out("file1"); // out и trunc установлены неявно
ofstream out2("file1", ofstream::out); // trunc установлен неявно
ofstream out3("file1", ofstream::out | ofstream::trunc);
// для сохранения содержимого файла следует явно задать режим app
ofstream app("file2", ofstream::app); // out установлен неявно
ofstream app2("file2", ofstream::out | ofstream::app);
```



ВНИМАНИЕ

Единственный способ сохранить существующие данные в файле, открытом потоком типа `ostream`, — это явно установить режим `app` или `in`.

Режим файла устанавливается при каждом вызове функции `open()`

Режим файла некоего потока может изменяться при каждом открытии файла.

```
ofstream out; // режим файла не установлен
out.open("scratchpad"); // неявно заданы режимы out и trunc
out.close(); // out закрыт, его можно использовать для другого файла
out.open("precious", ofstream::app); // режимы out и app
out.close();
```


Первый вызов функции `open()` не задает режим вывода явно; этот файл неявно открывается в режиме `out`. Как обычно, режим `out` подразумевает также режим `trunc`. Поэтому файл `scratchpad`, расположенный в текущем каталоге, будет усечен. Когда открывается файл `precious`, задается режим добавления. Все данные остаются в файле, а запись осуществляется в конец файла.



Режим файла устанавливается при каждом вызове функции `open()` явно или неявно. Когда режим не устанавливается явно, используется значение по умолчанию.

Упражнения раздела 8.2.2

Упражнение 8.7. Пересмотрите программу книжного магазина из предыдущего раздела так, чтобы вывод записывался в файл. Передайте имя этого файла как второй аргумент функции `main()`.

Упражнение 8.8. Пересмотрите программу из предыдущего упражнения так, чтобы добавить ее вывод в заданный файл. Запустите программу для того же выходного файла по крайней мере дважды и удостоверьтесь, что данные сохраняются.

8.3. Строковые потоки

Заголовок `sstream` определяет три типа, поддерживающие операции ввода-вывода в оперативной памяти; эти типы обеспечивают чтение или запись в строку, как будто она является потоком ввода-вывода.

Объект класса `istringstream` читает строку, объект класса `ostringstream` записывает строку, а объект класса `stringstream` читает и записывает строку. Подобно типам заголовка `fstream`, типы, определенные в заголовке `sstream`, происходят от типов, используемых заголовком `iostream`. Кроме унаследованных операций, типы, определенные в заголовке `sstream`, имеют дополнительные члены для работы со строками, связанными с потоком. Эти операции перечислены в табл. 8.5. Они могут быть выбраны для объектов класса `stringstream`, *строковых потоков* (`string stream`), но не других типов ввода-вывода.

Обратите внимание на то, что хотя заголовки `fstream` и `sstream` имеют общий интерфейс к заголовку `iostream`, никакой другой взаимосвязи у них нет. В частности, нельзя использовать функции `open()` и `close()` для объектов класса `stringstream`, а функцию `str()` нельзя использовать для объектов класса `fstream`.

Таблица 8.5. Операции, специфические для класса `stringstream`

<code>sstream strm;</code>	<code>strm</code> — несвязанный объект класса <code>stringstream</code> . <code>sstream</code> — это один из типов, определенных в заголовке <code>sstream</code>
<code>sstream strm(s);</code>	<code>sstream</code> содержит копию строки <code>s</code> . Этот конструктор является явным (см. раздел 7.5.4, стр. 383).
<code>strm.str()</code>	Возвращает копию строки, которую хранит объект <code>strm</code>
<code>strm.str(s)</code>	Копирует строку <code>s</code> в объект <code>strm</code> . Возвращает тип <code>void</code>

8.3.1. Использование класса `istringstream`

Класс `istringstream` зачастую используют тогда, когда некую работу следует выполнить со всей строкой и другую работу с отдельными словами в пределах строки.

Предположим, например, что имеется файл, содержащий список людей и номеров их телефонов. У одних людей есть только один номер, а у других несколько — домашний телефон, рабочий, мобильный и т.д. Наш исходный файл может выглядеть следующим образом:

```
morgan 2015552368 8625550123
drew 9735550130
lee 6095550132 2015550175 8005550000
```

Каждая запись в этом файле начинается с имени, затем следует один или несколько номеров телефонов. Для начала определим простой класс, представляющий исходные данные:

```
// по умолчанию члены являются открытыми; см. раздел 7.2 стр. 350
struct PersonInfo {
    string name;
    vector<string> phones;
};
```

Один член класса `PersonInfo` будет представлять имя человека, а вектор будет содержать переменное количество его номеров телефонов.

Наша программа будет читать файл данных и создавать вектор объекта класса `PersonInfo`. Каждый элемент вектора будет соответствовать одной записи в файле. Ввод обрабатывается в цикле, который читает запись, а затем извлекает имя и номера телефона каждого человека:

```
string line, word; // будут содержать строку и слово из ввода
vector<PersonInfo> people; // будет содержать все записи из ввода
// читать ввод по строке за раз, пока не встретится конец файла
// (или другая ошибка)
while (getline(cin, line)) {
    PersonInfo info; // создать объект для содержания данных записи
    istringstream record(line); // связать запись с читаемой строкой
```

```
record >> info.name;           // читать имя
while (record >> word)         // читать номер телефона
    info.phones.push_back(word); // и сохранить их
people.push_back(info);       // добавить эту запись в people
}
```

Здесь для чтения всей записи со стандартного устройства ввода используется функция `getline()`. Если вызов функции `getline()` успешен, то переменная `line` будет содержать запись из входного файла. В цикле `while` определяется локальный объект `PersonInfo` для содержания данных из текущей записи.

Затем с только что прочитанной строкой связывается поток `istreamstream`. Теперь для чтения каждого элемента в текущей записи можно использовать оператор ввода класса `istreamstream`. Сначала читается имя, затем следует цикл `while`, читающий номера телефонов данного человека.

Внутренний цикл `while` завершается, когда все данные в строке прочитаны. Этот цикл работает аналогично другим, написанным для чтения из объекта `cin`. Различие только в том, что этот цикл читает данные из строки, а не со стандартного устройства ввода. Когда строка прочитана полностью, встретившийся “конец файла” свидетельствует о том, что следующая операция ввода в объект `record` потерпит неудачу.

Внешний цикл `while` завершается добавлением в вектор только что обработанного объекта класса `PersonInfo`. Внешний цикл `while` продолжается, пока объект `cin` не встретит конец файла.

Упражнения раздела 8.3.1

Упражнение 8.9. Используйте функцию, написанную для первого упражнения 8.1.2 (стр. 404), для вывода содержимого объекта класса `istreamstream`.

Упражнение 8.10. Напишите программу для сохранения каждой строки из файла в векторе `vector<string>`. Затем используйте объект класса `istreamstream` для чтения каждого элемента из вектора по одному слову за раз.

Упражнение 8.11. Программа этого раздела определила свой объект класса `istreamstream` во внешнем цикле `while`. Какие изменения необходимо внести, чтобы определить объект `record` вне этого цикла? Перепишите программу, перенеся определение объекта `record` во вне цикла `while`, и убедитесь, все ли необходимые изменения внесены.

Упражнение 8.12. Почему в классе `PersonInfo` не использованы внутри-классовые инициализаторы?

8.3.2. Использование класса `ostringstream`

Класс `ostringstream` полезен тогда, когда необходимо организовать вывод небольшими частями за раз, не откладывая его на более позднее время. Например, могла бы возникнуть необходимость проверять и переформатировать номера телефонов, которые были прочитаны в коде предыдущего примера. Если все номера допустимы, необходимо переформатировать номера и вывести их в новый файл. Если у кого-нибудь будут недопустимые номера, то помещать их в новый файл не нужно. Вместо этого следует вывести сообщение об ошибке, содержащее имя человека и список его недопустимых номеров.

Поскольку нельзя включать для человека данные с недопустимыми номерами, мы не можем произвести вывод, пока не просмотрим и не проверим все их номера. Но можно “записать” вывод в оперативную память объекта класса `ostringstream`:

```
for (const auto &entry : people) {    // для каждой записи в people
    ostringstream formatted, badNums; // объекты создаются на каждом
                                    // цикле
    for (const auto &nums : entry.phones) { // для каждого номера
        if (!valid(nums)) {
            badNums << " " << nums; // строка в badNums
        } else
            // "запись" в строку formatted
            formatted << " " << format(nums);
    }
    if (badNums.str().empty()) // если плохих номеров нет
        os << entry.name << " " // вывести имя
        << formatted.str() << endl; // и переформатированные номера
    else // в противном случае вывести имя и плохие номера
        cerr << "input error: " << entry.name
        << " invalid number(s) " << badNums.str() << endl;
}
```

В этой программе подразумевается, что есть две функции, `valid()` и `format()`, которые проверяют и переформатируют номера телефонов. Интересная часть программы — использование строковых потоков `formatted` и `badNums`. Для записи в эти объекты используется обычный оператор вывода (`<<`). Но они действительно “пишут” строковые манипуляторы. Они добавляют символы к строкам в строковых потоках `formatted` и `badNums` соответственно.

Упражнения раздела 8.3.2

Упражнение 8.13. Перепишите программу номеров телефонов из этого раздела так, чтобы читать из именованного файла, а не из объекта `cin`.

Упражнение 8.14. Почему переменные `entry` и `nums` были объявлены как `const auto &`?

Резюме

Язык C++ использует библиотечные классы для обработки потоков ввода и вывода.

- Класс `iostream` обрабатывает ввод-вывод на консоль.
- Класс `fstream` обрабатывает ввод-вывод в именованном файле.
- Класс `stringstream` обрабатывает ввод-вывод в строки в оперативной памяти.

Классы `fstream` и `stringstream` связаны происхождением от класса `iostream`. Классы ввода происходят от класса `istream`, а классы вывода — от класса `ostream`. Таким образом, операции, которые могут быть выполнены с объектом класса `istream`, могут быть также выполнены с объектом класса `ifstream` или `istringstream`. Аналогично для классов вывода, происходящих от класса `ostream`.

Каждый объект ввода-вывода обладает набором флагов состояния, указывающих, возможен ли ввод-вывод через этот объект. Если произошла ошибка (например, встретился конец файла в потоке ввода), то состояние объекта окажется таково, что никакой дальнейший ввод невозможен, пока ошибка не будет исправлена. Библиотека предоставляет набор функций для установки и проверки этих состояний.

Термины

Класс `fstream`. Файловый поток, обеспечивающий чтение и запись в тот же файл. По умолчанию объект класса `ifstream` открывает файл одновременно в режимах `in` и `out`.

Класс `ifstream`. Файловый поток, читающий данные из файла. По умолчанию поток `ifstream` открывается в режиме `in`.

Класс `istringstream`. Строковый поток, читающий данные из строки.

Класс `ofstream`. Файловый поток, записывающий данные в файл. По умолчанию поток `ofstream` открывается в режиме `out`.

Класс `ostringstream`. Строковый поток, записывающий данные в строку.

Класс `stringstream`. Строковый поток, читающий и записывающий данные в строку.

Наследование (inheritance). Программное средство, позволяющее типу наследовать интерфейс другого типа. Классы `ifstream` и `istringstream` происходят от классов `istream` и `ofstream`, а класс `ostringstream` происходит от класса `ostream`. Более подробная информация о наследовании приведена в главе 15.

Режим файла (file mode). Флаги классов заголовка `fstream`, устанавливаемые при открытии файла и задающие способ его применения.

Строковый поток (string stream). Поточковый объект, читающий или записывающий данные в строку. Кроме возможностей, присущих классу `iostream`, классы строко-

вых потоков определяют перегруженную функцию `str()`. Вызов функции `str()` без аргументов возвращает строку, с которой связан объект строкового потока, а ее вызов со строковым аргументом свяжет строковый поток с копией этой строки.

Файловый поток (file stream). Потоковый объект этого класса позволяет читать и записывать данные в именованный файл. Кроме возможностей, присущих классу `iostream`, класс `fstream` обладает также функциями-членами `open()` и `close()`. Функция-член `open()` получает символьную строку в стиле C, которая содержит имя открываемого файла и необязательный аргумент, задающий режим. Функция-член `close()` закрывает файл, с которым связан поток. Ее следует вызвать прежде, чем может быть открыт другой файл.

Флаг состояния (condition state). Флаги и связанные с ними функции потоковых классов позволяют выяснить, пригоден ли данный поток для использования.